Universidad de San Andrés, Escuela de Ingeniería

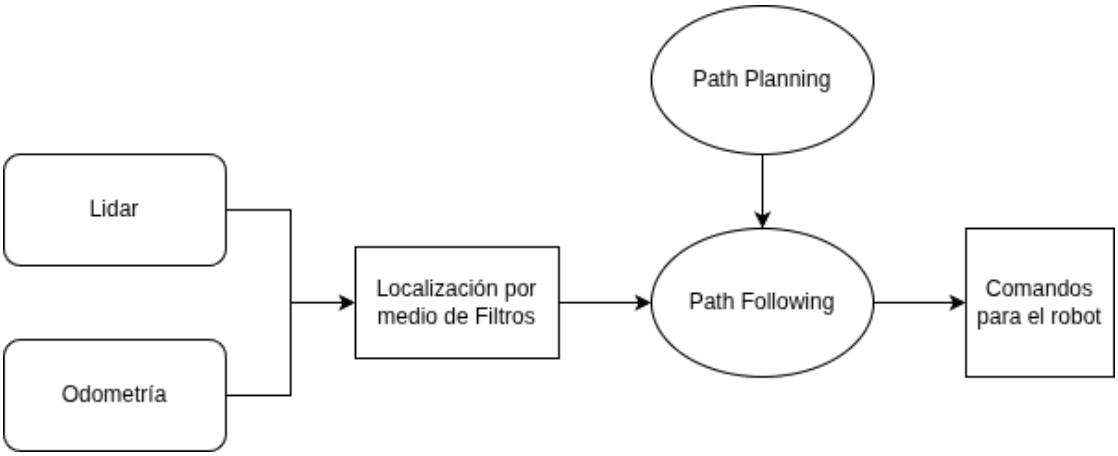
Parcial - Localización y Movimiento

I-402 - Principios de la Robótica Autónoma

Prof. Dr. Ignacio Mas, Tadeo Casiraghi y Bautista Chasco

Modo de entrega: Enviar por el Aula Virtual del Campus todo el código comentado y el informe (.pdf), todo en una sola carpeta comprimida.

Para este trabajo parcial estarán implementando el pipeline de localización completo. Corregirán la posición estimada del robot por medio de fusión de sensores y un mapa pre-existente. Con la posición corregida implementarán su algoritmo de path planning y pure pursuit para ejecutar el camino deseado en simulación, esta vez con la posición estimada en vez de la odometría cruda.



Por último evaluarán el desempeño de su algoritmo de localización con una grabación tomada con el robot real.

1. Filtro de Partículas

En esta primera sección implementaran un filtro de partículas.

1.1. Lanzamiento

Para correr el código debemos tener 3 terminales (no se olviden del colcon build y el source install/setup.bash)

- Terminal 1: `ros2 launch custom_map cafe.launch.py rviz:=False`
- Terminal 2: `ros2 launch tp_parcial particle_filter.launch.py`
- Terminal 3: `ros2 run teleop_twist_keyboard teleop_twist_keyboard`

Si siguieron todos los pasos correctamente habrán lanzado el mapa simulado, y se abrirá una ventana de RVIZ2. Si nos movemos con el teclado notarán que:

- Se carga el mapa sin campo de verosimilitud
- No hay partículas
- Se grafica el camino calculado en base a la odometría (azul), pero falta el camino (rojo) que calculamos con las partículas

1.2. Campo de Verosimilitud

Para poder navegar por el entorno y utilizar el sensor LIDAR debemos tener un modelo de sensado implementado. Deberán implementar un modelo de sensor LIDAR basado en el campo de verosimilitud. Esto implica que debemos tener un campo creado. Para eso utilizaremos el nodo de likelihood.py.

El nodo likelihood.py que se encuentra junto a la consigna se subscribe al tópico del mapa y debe publicar el campo de verosimilitud. Para ello deberán completar la función `map_callback` de manera que procese la información recibida y genere esta grila de probabilidades. El mensaje publicado debe contener la data en el campo data, que en ROS se espera como un array unidimensional de enteros de 8 bits con valores entre 0 y 100. La información del mapa llega en msg.data siguiendo un orden específico: comienza en la esquina inferior izquierda del mapa, avanza por cada fila de izquierda a derecha y luego continúa con la fila superior, hasta completar todo el mapa. En este arreglo, las celdas libres tienen valor 0, las ocupadas 100 y las desconocidas -1.

Deberán utilizar el valor de sigma que se guarda como parámetro (self.sigma). Si completaron correctamente el código al lanzar todo nuevamente deberán ver que se carga su mapa de verosimilitud.

1.3. Inicialización de las Partículas

Para tener un filtro de partículas es esencial primero tener partículas. Deberán agregar al esqueleto de nodo ya provisto (my_localization.py) la inicialización de sus partículas. La cantidad de partículas será definida por el parámetro externo num_particles. Para asignar la pose inicial deberán agregar una subscripción a pose inicial de RVIZ (como en el TP2) y en base a eso crear un campo de partículas entorno a la pose indicada. Como no conocemos la posición perfecta y no se puede confiar en absoluto en lo que se indica en RVIZ se recomienda que inicie las partículas con cierta dispersión en X, Y y Tita.

1.4. Movimiento de Partículas

Ahora que tenemos un campo funcional y partículas, implementaremos el movimiento. Deberán completar la función:

`odom_callback`

Esta función ya publica el camino calculado con odometría. Ustedes deberán implementar el código que calcule la nueva posición de las partículas. Pueden utilizar como dato el valor de self.last_odom que guarda la odometría pasada. La actualización de las partículas deberá ser con el modelo de movimiento correcto y aleatorio para cada partícula.

Si con las 1000 partículas default corre muy lento o no se localiza correctamente puede probar cambiando la cantidad de partículas. Para eso puede correr el launch file de la siguiente manera:

```
ros2 launch tp_parcial particle_filter.launch.py num_particles:=500
```

1.5. Sensado y Resampleo

Por último deberán implementar el modelo de sensor y el resampleo de partículas. Para ello deberán completar la función:

`scan_callback`

Cada vez que se reciba data del lidar deberán implementar el cálculo de los pesos de cada partícula y luego resamplear para quedarse con una mejor distribución de partículas sanas. Para eso tienen a su disposición:

- **data**: datos del lidar en formato scan (Ver documentacion de ROS sobre tipo de dato LaserScan).
- **self.map_data**: Es el mensaje crudo del campo de verosimilitud. Pueden consultar la documentacion de ROS sobre tipos de dato OccupancyGrid.
- **self.grid**: Es la representación como matriz de numpy del campo de verosimilitud. Importante:

- La grilla se indexa como grid[y, x], primero fila (eje Y) y luego columna (eje X)
- La celda (0,0) corresponde a la esquina inferior izquierda del mapa en coordenadas de ROS.

Tengan cuidado al resamplear de tomar los recaudos necesarios (por ejemplo, hacer deepcopy para que no se vinculen las listas)

Tenga en cuenta que la terna del LIDAR esta corrida 0.043 metros en x, 0.110 metros en z, y rotada 180 grados respecto a la terna del robot. Tendrán que tener eso en cuenta al momento de sensar.

Si se completó todo correctemante verán que, al mover el robót con el nodo del teclado, ahora las partículas convergen a la posición **real** del robot, y que el camino rojo sigue el camino verde.

1.6. Publicación de la pose estimada

Ahora que tienen un filtro en funcionamiento hay que elegir cómo publicar la pose corregida. Deberán implementar un publicador de ros que publique la pose corregida. Pueden decidir si publicarla siempre que se recibe un valor del lidar, a una frecuencia determinada fija con un timer, o algún otro método de publicación. También debeerán decidir si la pose es un promedio de las partículas, un promedio pesado, la mejor partícula, o cualquier otro criterio que les parezca pertinente.

1.7. Path Planning, Path Following y Análisis

Con el filtro de partículas en funcionamiento, vuelva a implementar su código del TP2 pero ahora utilizando la pose corregida en vez de la calculada en base a odometría cruda. Evalue y responda a las siguientes preguntas:

1. ¿Afecta al desempeño de su algoritmo el valor del sigma del campo de verosimilitud? ¿En caso que sí lo afecte, que consecuencias nota? ¿Qué ocurre si es muy chico/grande? Puede cambiarlo fácilmente con el siguiente comando:

   ```
   ros2 launch tp_parcial particle_filter.launch.py sigma:=2.0
   ```

2. ¿Cómo afecta la dispersión inicial y la cantidad de partículas al funcionamiento del filtro? ¿Cómo afecta al desempeño una dispersión inicial muy chica? ¿Una muy grande? ¿Cómo interactúa el tamaño de la dispersión inicial con la cantidad de partículas?

3. ¿Cómo afecta al desempeño del filtro la cantidad de partículas? ¿Hay un valor mínimo luego del cual no converge? ¿Porqué?

4. ¿Logra su robot recorrer correctamente el entorno ahora que está filtrando la posición?

5. ¿Se le ocurre alguna manera de utilizar el campo de verosimilitud para hacer más seguro el path planning? Puede utilizarlo para informar el algoritmo que decide que recorrido tomar?

2. EKF

Para esta sección del trabajo deberán implementar un filtro EKF con un sensor de landmarks simulado.

2.1. Proceso de lanzamiento de nodos

Para correr todo el código debemos tener 3 terminales (no se olviden del colcon build y el source install/setup.bash). En primer instancia, utilizaremos 3 (hay que respetar este orden de lanzamiento):

- Terminal 1: `ros2 launch custom_map cafe_features.launch.py rviz:=False`
- Terminal 2: `ros2 launch tp_parcial ekf.launch.py`
- Terminal 3: `ros2 run teleop_twist_keyboard teleop_twist_keyboard`

Si siguieron todos los pasos correctamente deberían ver una terminal de RVIZ con el mapa y los posibles landmarks.

2.2. Prediccion por EKF

Ahora tendrán que implementar un paso de predicción por EKF. Para eso tendrán que modificar el nodo de ekf.py para que haga lo siguiente:

1. Esperar al input de RVIZ de posición inicial y crear un belief interno de la posición del robot en base a eso. Por el momento pueden asumir que la covarianza inicial del robot es de 0.5 metros en x e y, y 10 grados (recuerde trabajar en radianes) de orientación.
2. Dentro del callback de odometría ejecutar el proceso de predicción por EKF
3. Publicar una pose con covarianza para poder verlo en RVIZ.

El nodo deberá mantener siempre un valor de belief actualizado (a futuro la corrección cambiará el valor del belief).
Para calcular la predicción deben:

1. Calcular la matriz jacobiana G_t de la función de movimiento g sin ruido.

2. Implementar el paso de predicción usando el jacobiano G_t calculado en el punto anterior. Asumir que el ruido del modelo de movimiento esta definido por:

$$Q_t = \begin{pmatrix} 0{,}02 & 0 & 0 \\ 0 & 0{,}02 & 0 \\ 0 & 0 & 0{,}02 \end{pmatrix}$$

Si hizo todo correctamente es de esperar que la covarianza aumente a medida que se mueve el robot. Dado a que todavia no hemos corregido con el sensor de landmarks solo debe crecer.

2.3. Paso de corrección EKF

Ahora tendrán que implementar un paso de corrección por EKF. Para eso tendrán que nuevamente modificar el nodo de ekf.py tal que:

1. Se subscriba al tópico `/landmarks` de tipo de dato **PoseArray** con QoSDurabilityPolicy: TRANSIENT_LOCAL y QoSReliabilityPolicy: RELIABLE. Así recibirán el dato de la posición de todos los landmarks.

2. Se subscriba al tópico `/observed_landmaks` de tipo de dato **PoseArray**. Así recibirán el dato de los landmarks observados. Este array de `Pose2D` es de largo n, siendo n la cantidad total de landmarks. Cuando se detecta un landmark se publica su pose estimada **local** en x e y (pose.position.x y pose.position.y). El orden de los landmarks no corresponde con el mismo orden de la lista total de landmarks. Deberán procesar los datos para hacer una asosiación de landmark observado a landmark posible. Los landmarks que no son visibles son reportados como 0 en x y 0 en y.

3. Haga un update del belief corrigiendo la posición con los datos de los landmarks.

Para hacer el paso de corrección tendrá que:

1. Calcular la matriz jacobiana H_t de la función de medición h en base a los landmarks.

2. Implementar el paso de corrección del filtro EKF en el callback del sensor usando el jacobiano H_t. Asumir que el desvío estándar de la medición de rango es de 0.06 metros, y del ángulo 0.07 radianes.

3. Publicar el belief actualizado

Si hicieron todo correctamente deberían ver que se corrige la posición para estar más cerca de la posición real.

2.4. Path Planning, Path Following y Análisis

Con el filtro ekf en funcionamiento, vuelva a implementar su código del TP2 pero ahora utilizando la pose corregida en vez de la calculada en base a odometría cruda.
Evalue y responda a las siguientes preguntas:

1. ¿Afecta al desempeño de su algoritmo la covarianza inicial asumida para el robot? ¿Qué ocurre si es muy chica/grande?
2. ¿Cómo afecta la selección de Q al funcionamiento del filtro? ¿Cómo afecta al desempeño asumir errores muy grandes o muy chicos?
3. ¿Cómo afecta el fov de landmarks al funcionamiento? ¿Qué ocurre si es muy acotado? ¿Qué ocurre si es 360? Puede cambiarlo de esta manera:

   ```
   ros2 launch custom_map cafe_features.launch.py fov:=360.0
   ```

4. ¿Logra su robot recorrer correctamente el entorno ahora que está filtrando la posición?
5. ¿En secciones donde hay pocos landmarks (como por ejemplo entre la barra y la pared externa del lado donde no está la puerta de la cocina) como se comporta el filtro?

3. ROSBAG

Un rosbag es una grabación de ROS donde se guardan todos los topicos seleccionados, sus mensajes y los tiempos de cada mensaje. Estas grabaciones pueden ser reproducidas de tal manera que los tópicos se publiquen en tiempo y forma. Con esto podemos entonces probar nuestros algoritmos sobre datos reales del robot sin tener que tener el robot físico al lado. La unica contraparte es que, al ser grabaciones, no podemos afectar al movimiento ya hecho o a los datos ya tomados.
Como parte final de este trabajo práctico parcial tendrán que probar su algoritmo de filtro de partículas con los datos del robot real. A diferencia de la simulación no tenemos un dato de posición ground truth ya que estamos trabajando con el robot real. Pueden econtrar y descargarse el rosbag desde este link.
Para eso ejecuten lo siguiente:

- `ros2 launch tp_parcial particle_filter.launch.py odom_topic:=/odom num_particles:=1000`
- `ros2 launch custom_map real.launch.py`
- `ros2 bag play ruta/a/tu_bag --clock`

Evalúen si su filtro de partículas funciona para los datos reales. Es posible que tengan que modificar sus parámetros para ajustarlos al caso real.
Cómo referencia, el robot debería terminar cerca de donde comenzó.
Para el informe comente sobre el funcionamiento de su filtro con datos reales